

# Luis San Martín

Ingeniero de Visión Computacional · Desarrollador de Software

Concepción, Chile | +56 9 7587 8696 | sanmaglass@gmail.com  
sanmaglass.github.io | linkedin.com/in/sanmartinluis | github.com/sanmaglass

## PERFIL

Más de tres años construyendo software que queda funcionando en producción. Entreno y despliego **modelos de visión computacional** —detección, clasificación y seguimiento de objetos— sobre cámaras en operación real, y construyo los sistemas de datos que integran y reportan esa información sin intervención. El entrenamiento va en la nube y la inferencia queda corriendo en el hardware del cliente. También dejo **agentes corriendo solos** —reportes, alertas e indicadores por mensajería— integrando modelos de lenguaje donde aportan y evitándolos donde no hacen falta. Trabajo directo con los dueños de cada operación, desde el requerimiento hasta el sistema andando. Egresado de Ingeniería Civil en Automatización, Universidad del Bío-Bío.

## HABILIDADES TÉCNICAS

|                            |                                                                                                                                                                                                                                                            |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Visión computacional       | PyTorch · YOLOv8 · OpenCV · detección y clasificación de objetos · seguimiento multiobjeto (SORT) · <i>fine-tuning</i> sobre datasets propios · etiquetado de imágenes en terreno · inferencia en tiempo real                                              |
| Entrenamiento y despliegue | Entrenamiento en la nube (Google Colab) · inferencia local <i>on-premise</i> sobre GPU del cliente · despliegue en el borde (ESP32-S3) · cámaras IP sobre RTSP · evaluación por mAP, precisión y <i>recall</i>                                             |
| Agentes e IA generativa    | Agentes conversacionales sobre WhatsApp y Telegram en operación · integración con APIs de LLM (Groq, Anthropic) · clasificación de intención · clasificador propio de n-gramas sin dependencia de API · observabilidad de LLM con Langfuse y OpenTelemetry |
| Lenguajes                  | Python · SQL · TypeScript / JavaScript                                                                                                                                                                                                                     |
| Datos y reportería         | PostgreSQL (modelado, migraciones versionadas, control de acceso por fila) · Firebird · SQLite · ETL e integración de sistemas sin API · web scraping · dashboards en tiempo real · reportes automatizados y alertas por umbral                            |
| Infraestructura            | Linux · Docker · Git · GitHub Actions · servidores en la nube · procesos programados y monitoreo                                                                                                                                                           |

## EXPERIENCIA

### Desarrollador de Software — Distribuidora El Maravilloso

— Integré el punto de venta —base Firebird, **sin API**— con una base PostgreSQL en la nube: mapeé el esquema origen y construí la sincronización, hoy con **11.900 ventas y 40.000 líneas de producto** consolidadas (≈78 ventas diarias).

— **ETL desatendido** que extrae los documentos tributarios desde el portal del SII y calcula los indicadores mensuales de impuestos: **305 facturas de compra** importadas y clasificadas sobre 87 proveedores.

— **Control de calidad de datos:** cruce de tres fuentes —documentos tributarios, pagos electrónicos y ventas del punto de venta— para detectar ingresos duplicados y descuadres de caja: **283 alertas** emitidas sin alterar las cifras del cliente.

— **Agentes en operación** sobre WhatsApp y Telegram: entregan el cierre de ventas y los indicadores tributarios del mes, y avisan por umbral cuando algo se descuadra. Construí un **clasificador de intención propio por n-gramas** (82% de acierto, tolerante a errores de tipeo) que resuelve la clasificación **sin llamar a ninguna API**, y dejé las rutas de IA instrumentadas con Langfuse.

— Reportería en tiempo real, informe ejecutivo mensual y diagnóstico de **incidentes de integridad en producción** (pérdida de registros por una regla de limpieza mal acotada, desfase de relojes entre origen y nube), recuperando los datos afectados.

**Stack:** Python · SQL · PostgreSQL · Firebird · JavaScript · Linux · procesos programados

Nov 2025 – Presente · Concepción

- Migré la operación desde la plataforma comercial que usaban —~170 alumnos activos—: extraje su base de datos completa por **web scraping**, la normalicé y trasladé clientes e historial sin pérdida.
- Diseñé el esquema en **PostgreSQL**: multi-sede con control de acceso por fila y migraciones versionadas, para **2 sedes**.
- Construí el panel de administración y su reportería: asistencia, ocupación por sede y horario, ingresos y liquidación de comisiones de **14 profesionales** sobre una agenda de 768 sesiones.

Stack: PostgreSQL · SQL · Python · TypeScript

Ingeniero de Visión Computacional — St. Andrews · Mussels

Jun 2024 — Jul 2025 · Puerto Montt Chiloé

- Entrené y desplegué un sistema multicámara de **detección, clasificación y conteo en 3 estaciones de empaque** sobre una línea de **13.000 kg diarios**, en el mayor productor de mejillones del mundo.
- **Seguimiento multiobjeto (SORT)** para mantener el identificador de cada paquete entre cuadros y evitar el doble conteo cuando la cinta se detiene o el operario interviene.
- Modelo de **conteo de larvas** que bajó el análisis manual de **45 minutos a segundos**: 696 individuos en 2,93 s con 85% de acierto, entrenado sobre un **dataset propio etiquetado en la planta**.
- Robusto ante iluminación cambiante, cámaras que se mueven y superficies mojadas. **Operó una temporada completa sin intervención manual**, con la inferencia corriendo en los equipos de la propia planta.

Stack: Python · PyTorch · YOLOv8 · OpenCV · SORT · RTSP sobre cámaras IP

PROYECTOS DE VISIÓN COMPUTACIONAL E IA

Detección de casco sobre cámara embebida — Ewood · movilidad eléctrica

2026

- **Fine-tuning** de YOLOv8 a 3 clases (scooter, con casco, sin casco) sobre **12.321 imágenes** y 80 épocas: **mAP@50 de 85,5%**, precisión 85,0% y *recall* 80,9%.
- Inferencia en vivo a **9,6 cuadros por segundo** sobre una **cámara embebida XIAO ESP32-S3 Sense**, no sobre una cámara de circuito cerrado: la restricción era que el hardware fuera montable en el scooter.

Stack: PyTorch · YOLOv8 · OpenCV · Python · ESP32-S3

Analítica de flujo de personas en retail — con cliente, en producción

Feb 2026 — Presente

- Conteo de entrada y salida sobre las cámaras de circuito cerrado que el local ya tenía, mapas de calor de tráfico acumulado y alertas por cruce de línea configurable, sin hardware adicional.

Stack: Python · YOLOv8 · OpenCV

Detección de elementos de protección personal (EPP) — proyecto de título

2026 — en curso

- Detección de casco, chaleco y demás EPP en faena industrial. El aporte central es el **dataset propio, etiquetado en faena real**, no bancos públicos que no representan las condiciones de trabajo.

Stack: PyTorch · YOLOv8

FORMACIÓN

Ingeniería Civil en Automatización — Universidad del Bío-Bío

Egresado 2024 · Titulación 2026

IdiomasEspañol (nativo) · Inglés (lectura técnica)

DisponibilidadInmediata · Gran Concepción, remoto o híbrido

Casos con imágenes y métricas en **sanmaglass.github.io**. Los repositorios son privados por tratarse de código en operación de clientes; disponibles a revisión bajo acuerdo.